

OFFICIAL PROJECT ARCHITECTURE & MASTER GUIDE

NormMix AI: An Attention-Based Sequence-to-Sequence Framework for Telugu-English Code-Mixed Text Normalization

A Comprehensive Pedagogical Guide to the Codebase Anatomy, Computer Directory Taxonomy, Bottom-Up Learning Curriculum, and Viva Defense Presentation Guide.

PROJECT DOMAIN	COURSE / CONTEXT
Indic NLP, Computational Linguistics & Deep Learning	Advanced Artificial Intelligence & Neural Networks (AAN)
HARDWARE ACCELERATION	CORE ARCHITECTURE
NVIDIA GeForce RTX 4060 Laptop GPU (8GB GDDR6)	2-Layer BiGRU + Scaled Dot Attention + Copy Gate
INFERENCE LATENCY	PRODUCTION DELIVERABLES
~8.2 ms (CUDA FP16 Mixed Precision)	FastAPI Backend, Web Studio SPA, Chrome Extension

Key Benchmark Breakthrough: Achieved an **8x error reduction** over baselines, reaching a Character Error Rate (CER) of **5.37%**, chrF of **91.29**, and BLEU of **79.84**, with **100.0% English loanword preservation**.

1. Executive Summary & Problem Formulation

In Indian digital communication (WhatsApp, Instagram, X/Twitter, YouTube), Telugu speakers predominantly communicate in **Tanglish**—colloquial Telugu written in the Latin alphabet intermingled with English loanwords, technical terminology, abbreviations, and informal chat syntax.

The Core Challenge: Standard Machine Translation systems (Google Translate, Bing) fail catastrophically on code-mixed text because they attempt to phonetically transliterate English loanwords into Telugu script (e.g., college → కాలేజ్ , interview → ఇంటర్వ్యూ). This degrades readability and downstream comprehension.

System Objective: Given an input sequence $X = (x_1, \dots, x_{|T_x|})$, the model must:

1. Accurately transliterate colloquial Romanized Telugu morphemes to pristine Telugu Unicode script (e.g., `nakula` → నాకు ఇవాళ).
2. Dynamically recognize embedded English loanwords and route them through a **Pointer-Generator Copy Gate**, retaining Latin characters with 100% fidelity.
3. Provide multi-modal generation: Standard Normalized, All Telugu Script, All Romanized Tanglish, and Pure English Semantic Translation.

2. Master Directory Taxonomy: What Every Folder on Your Computer Does

The following table defines every single folder in the project repository (`c:\projects\NormMix`), explaining its exact purpose, key contents, and role in the system lifecycle.

Directory / File	Purpose & Functionality	Key Files Inside	Executed By
<code>src/</code>	Core Neural & NLP Library: Contains all deep learning architectures, attention implementations, tokenizers, preprocessing, and training loops.	<code>config.py</code> , <code>attention/</code> , <code>models/</code> , <code>pipeline/</code>	Imported across project
<code>src/pipeline/</code>	OmniProcessor Multi-Modal Engine: Master orchestrator that performs script modality detection and coordinates 4-way translation.	<code>omni_processor.py</code>	API & Web Studio
<code>src/models/</code>	Deep Learning Architecture: Implements 2-Layer BiGRU with Copy Gate, Seq2Seq Transformer, Rule Baseline, and Beam Search.	<code>rnn_seq2seq.py</code> , <code>decoding.py</code> , <code>transformer.py</code>	Training & Inference
<code>src/attention/</code>	Attention Mechanisms: Math implementations of Scaled Dot-Product, Bahdanau (additive), and	<code>attention.py</code>	Attached inside BiGRU

Directory / File	Purpose & Functionality	Key Files Inside	Executed By
	Luong (multiplicative) attention.		
src/preprocessing/	Linguistic Preprocessing: Unicode NFC normalization, repeat clamping, URL/@mention protection, and Char n-gram Language ID.	text_cleaning.py , langid.py , translit_te.py	Pipeline & Augmentation
src/datasets/	Data Loaders & Splitting: Adapters for Google Dakshina and Aksharantar, zero-leakage hash split, and PyTorch DataLoaders.	splits.py , pipeline.py , torch_data.py	Data Prep & Training
src/tokenization/	Tokenization Subsystem: Compact character-level tokenizer (~180 tokens) and SentencePiece subword tokenizer.	char_tokenizer.py , spm_tokenizer.py	Embedding layers
src/training/	PyTorch Training Engine: Handles FP16 mixed precision, AdamW optimizer, ReduceLROnPlateau scheduler, and checkpointing.	loop.py	scripts/train.py
src/evaluation/	Benchmark Evaluation: Computes CER, WER, chrF, BLEU, and fine-grained English vs Telugu token accuracy.	metrics.py , evaluate.py	scripts/evaluate.py
api/	FastAPI REST Microservice: Asynchronous REST backend providing endpoints for normalization, translation, romanization, and feedback.	main.py , schemas.py , model_registry.py	uvicorn on port 8000
data/processed/	Ready-to-Train Dataset Splits: 25,803 zero-leakage training pairs, 64k rule baseline dictionary, and held-out OOV test slices.	train.jsonl , test.jsonl , rule_baseline.json	PyTorch DataLoader
experiments/checkpoints/	Trained Model Weights: Serialized PyTorch weight files (best.pt , last.pt) for Production SOTA and baseline models.	production_sota/best.pt	ModelRegistry
web/	Enterprise Web Translation Studio: Single-	index.html	Web Browser

Directory / File	Purpose & Functionality	Key Files Inside	Executed By
	page application with real-time phonetic suggestions, Web Speech dictation, and audio TTS.		
chrome-extension/	Manifest V3 Browser Extension: In-place text normalization on WhatsApp Web, Twitter, LinkedIn, and Gmail via context menu.	manifest.json , popup/ , content/	Google Chrome / Edge
scripts/	CLI Automation Tools: Standalone executable scripts for dataset downloading, model training, evaluation, and slide generation.	train.py , evaluate.py , prepare_data.py	Terminal CLI
tests/	PyTest Test Suite: 58 unit tests covering tensor shapes, attention masking, translation fidelity, and API endpoints.	test_models_forward.py , test_api.py	pytest runner
docs/	Academic Reports & Presentations: Master technical documentation, PowerPoint presentation (presentation.pptx), and PDF reports.	project_report.pdf , 00_master_report.md	Academic Review
.venv/ & .venv-gpu/	Python Virtual Environments: Isolated runtime environments containing Python binaries, PyTorch with CUDA 12.8, and dependencies.	python.exe , site-packages/	System execution

3. Bottom-Up Pedagogical Learning Path (Level 1 to Level 9)

To understand every layer of this system conceptually and mathematically, follow this structured bottom-up learning curriculum:

```

Level 9: Full-Stack Web Studio, Chrome Extension & Active Learning Loop
▲
Level 8: OmniProcessor Multi-Modal Engine & Semantic Translation
▲
Level 7: Inference, Autoregressive Decoding & Vectorized Beam Search
▲
Level 6: Loss Functions, Teacher Forcing & CUDA FP16 Training Dynamics
▲
Level 5: Attention Mechanisms Compared: Bahdanau vs Luong vs Scaled Dot
▲
Level 4: Neural Architecture: 2-Layer BiGRU + Pointer-Generator Copy Gate
▲
Level 3: Character-Level Tokenization & Representation Theory
▲
Level 2: Data Engineering, Zero-Leakage Hashing & Synthetic Augmentation
▲
Level 1: Linguistic Foundations: Code-Mixing, Script Divergence & Tanglish

```

Level 1: Linguistic Foundations of Tanglish

Telugu is an abugida script with 56 primary characters. Tanglish uses the 26-letter Latin alphabet to represent these sounds phonetically. Because there is no official standardized spelling, users write with huge variations:

- **Vowel elongation:** *chala* → *chaala*, *chaalaa* ("very").
- **Aspiration omission:** *dhanyavadalu* → *danyavadalu* ("thanks").
- **Intra-sentential loanwords:** English nouns/verbs are embedded directly into Telugu grammar (e.g., *college lo, interview undi*).

Level 2: Data Engineering & Zero-Leakage Hashing

Standard random splits leak identical sentences across train and test, producing artificially inflated metrics. We implemented deterministic SHA-1 bucket hashing on target sentence groups:

```

def bucket(group: str, salt: str = "tecm-v1") → int:
    return int(hashlib.sha1(f"{salt}:{group}".encode()).hexdigest()[:8], 16) % 10000

def assign_split(group: str, valid_frac: float = 0.1, test_frac: float = 0.1) → str:
    b = bucket(group) / 10000
    if b < test_frac: return "test"
    if b < test_frac + valid_frac: return "valid"
    return "train"

```

Held-out vocabulary words (e.g., `interview`, `charger`, `canteen`) are quarantined strictly into `test_unseen_words.jsonl` to test true out-of-domain generalization.

Level 3: Character-Level Tokenization

Subword tokenizers (BPE) break under informal character elongation, splitting unseen variants into rare subwords or `<unk>`. A character tokenizer maps ~180 characters (Latin, Telugu Unicode 0x0C00–0x0C7F, digits, punctuation), reducing the out-of-vocabulary rate to **0.0%**.

Level 4: Neural Architecture: BiGRU + Pointer-Generator Gate

The encoder is a 2-layer Bidirectional GRU yielding concatenated states $\mathbf{h}_i = [\vec{\mathbf{h}}_i; \overleftarrow{\mathbf{h}}_i] \in \mathbb{R}^{2d_h}$. At each decoder step, the **Pointer-Generator Copy Gate** calculates:

```
p_gen = torch.sigmoid(self.p_gen_layer(torch.cat([feat, e], -1)))
vocab_probs = F.softmax(logits, dim=-1)
p_final = p_gen * vocab_probs
p_final.scatter_add_(1, src_mapped, (1.0 - p_gen) * w)
logits = torch.log(p_final.clamp(min=1e-12))
```

When encountering an English loanword, $p_{\{\text{gen}\}} \rightarrow 0$, copying the Latin character directly from the input sequence!

Level 5: Attention Mechanisms Compared

We evaluated three distinct attention architectures in `src/attention/attention.py`:

- **Bahdanau (Additive):** $e_{t,i} = v^T \tanh(W_q s_{t-1} + W_k h_i)$. Evaluated before RNN step.
- **Luong (General):** $e_{t,i} = s_t^T W_a h_i$. Evaluated after RNN step with input feeding.
- **Scaled Dot-Product (Winner):** $e_{t,i} = \frac{(W_q s_t)^T (W_k h_i)}{\sqrt{d}}$. Scaling prevents gradient vanishing in softmax.

Level 6: Loss Functions & CUDA FP16 Training Dynamics

Trained using Cross-Entropy with Label Smoothing ($\epsilon = 0.05$) to prevent over-confidence. Mixed-precision FP16 via `torch.cuda.amp.autocast()` cut memory in half and doubled speed on the NVIDIA RTX 4060 GPU.

Level 7: Inference & Vectorized Beam Search

At inference time, greedy decoding is prone to local minima. We implemented vectorized beam search ($K=4$) with length normalization penalty $\alpha = 1.0$: $\text{score}_{\text{norm}}(Y) = \frac{\sum_{t=1}^T \log P(y_t | y_{<t})}{T}$

Level 8: OmniProcessor & 4-Way Multi-Modal Generation

The OmniProcessor analyzes the input string and synthesizes 4 simultaneous outputs:

1. **Standard Normalized:** Telugu in Telugu script, English loanwords preserved in Latin.
2. **All Telugu Script:** Complete transliteration of loanwords into Telugu script (ఇంటర్వ్యూ).
3. **All Romanized Tenglish:** Phonetic transliteration back to clean Latin alphabet.
4. **Pure English Translation:** Word-by-word glossing and grammatical elevation to natural English.

Level 9: Full-Stack Serving & Active Learning

The FastAPI backend connects to `web/index.html` (supporting Web Audio TTS, speech recognition, and virtual Telugu keyboard) and `chrome-extension/` (Manifest V3 context menu). Every user thumbs-up/down is appended to `data/feedback.jsonl` for active learning model fine-tuning.

4. Codebase Anatomy: Where to Find Every Line of Code

Use this master table to immediately locate the exact file, class, method, and line numbers in the codebase:

Concept / Subsystem	File Path	Class / Function / Symbol	Line Numbers
Unicode NFC & Repeat Clamping	<code>src/preprocessing/text_cleaning.py</code>	<code>normalize_unicode()</code> , <code>clean_text()</code>	Lines 38–58
URL & Hashtag Protection	<code>src/preprocessing/text_cleaning.py</code>	<code>split_protected()</code>	Lines 60–82
Language ID (Char n-gram)	<code>src/preprocessing/langid.py</code>	<code>CharNgramLID.predict_proba()</code>	Lines 49–75
Phonetic Transliteration Rules	<code>src/preprocessing/translit_te.py</code>	<code>romanize_word()</code> , <code>CONSONANTS</code>	Lines 19–85

Concept / Subsystem	File Path	Class / Function / Symbol	Line Numbers
Character Tokenizer	src/tokenization/char_tokenizer.py	CharTokenizer.encode() , decode()	Lines 15–54
Scaled Dot-Product Attention	src/attention/attention.py	ScaledDotProductAttention.forward()	Lines 69–84
Bahdanau & Luong Attention	src/attention/attention.py	BahdanauAttention , LuongAttention	Lines 27–67
BiGRU Encoder & Hidden Bridge	src/models/rnn_seq2seq.py	RNNSeq2Seq.encode()	Lines 69–84
Pointer-Generator Copy Gate	src/models/rnn_seq2seq.py	p_gen_layer , scatter_add_	Lines 116–122
Vectorized Beam Search	src/models/decoding.py	beam_search()	Lines 15–68
Zero-Leakage Hash Splitting	src/datasets/splits.py	bucket() , assign_split()	Lines 10–48
PyTorch Collate & Padding	src/datasets/torch_data.py	collate_fn() , CodeMixedDataset	Lines 35–80
Training Loop & FP16 Scaler	src/training/loop.py	train_model()	Lines 35–180
OmniProcessor Multi-Modal Engine	src/pipeline/omni_processor.py	OmniProcessor.process()	Lines 1873–1920
Pure English Translation	src/pipeline/omni_processor.py	translate_to_pure_english()	Lines 779–945
Pure Telugu Script Converter	src/pipeline/omni_processor.py	translate_to_pure_telugu()	Lines 1602–1740
FastAPI REST Endpoints	api/main.py	/omni/process , /normalize , /translate	Lines 102–155
Model Registry & GPU Cache	api/services/model_registry.py	ModelRegistry.get() , normalize()	Lines 25–95
Web Translation Studio UI	web/index.html	Speech API, Keyboard Drawer, Suggestions	Lines 1–850
Chrome Extension Service Worker	chrome-extension/background/service_worker.js	Context Menu & Background Listener	Lines 1–45

Concept / Subsystem	File Path	Class / Function / Symbol	Line Numbers
Chrome In-Page Replacer	chrome-extension/content/content_script.js	DOM selection replacement	Lines 1–75

5. End-to-End System Execution Flow: Step-by-Step Lifecycle

Here is what happens when a user types: `naku ivala college lo important interview undi` :

- Input Ingestion:** The Web Studio (or Chrome Extension) sends an HTTP POST request to `/omni/process` with payload `{"text": "naku ivala college lo important interview undi"}`.
- Modality Detection:** `detect_modality()` in `src/pipeline/omni_processor.py` scans characters. It detects Romanized characters mixed with English lexicon tokens (`college` , `interview`) and classifies it as `code_mixed`.
- Model Registry Lookup:** `ModelRegistry` retrieves the active `production_sota` checkpoint cached in GPU VRAM (NVIDIA RTX 4060).
- Character Tokenization:** `CharTokenizer` converts input characters into a 1D integer tensor $[1, 48, 12, 55, \dots]$.
- BiGRU Encoding:** The 2-Layer BiGRU encoder processes the sequence forward and backward, producing hidden state tensor $\mathbf{H} \in \mathbb{R}^{B \times T_x \times 384}$.
- Attention & Pointer-Generator Decoding:**
 - For Telugu tokens (*naku* → నాకు, *ivala* → ఇవాళ): Scaled Dot-Product Attention focuses on phonetic source characters, $p_{\{\text{gen}\}} \approx 0.95$, and the model samples from Telugu Unicode vocabulary.
 - For English loanwords (*college*, *interview*): The model detects Latin character sequences, $p_{\{\text{gen}\}} \approx 0.02$, and the gate copies Latin characters directly from the source sequence!
- Beam Search Decoding:** Vectorized beam search ($K=4$) ranks hypotheses with length penalty $\alpha=1.0$, producing the best sequence: నాకు ఇవాళ college లో important interview ఉంది .
- Multi-Modal Generation:** The OmniProcessor synthesizes 4 options:
 - Standard Normalized:* నాకు ఇవాళ college లో important interview ఉంది
 - All Telugu Script:* నాకు ఇవాళ కాలేజ్ లో ఇంపార్టెంట్ ఇంటర్వ్యూ ఉంది
 - All Tanglish:* naaku ivaala college lo important interview undi
 - Pure English Translation:* "I have an important interview at college today"
- Response & Audio TTS:** The response is returned in **~8.2 ms**. The Web Studio renders cards, triggers Web Speech TTS audio, and displays real-time suggestions.
- Active Learning Feedback:** User rating (🍌 / 🍋) is saved to `data/feedback.jsonl`.

6. Academic Presentation & Viva Defense Master Guide

6.1 The 30-Second Elevator Pitch

"Our project, **NormMix AI**, addresses the major NLP challenge of Indian social media communication where Telugu and English are heavily code-mixed into informal 'Tanglish'. Traditional machine translation models suffer catastrophic loanword corruption, turning English words into unreadable phonetic characters. To solve this, we engineered a custom **2-Layer Bidirectional GRU with Scaled Dot-Product Attention** coupled with a **Pointer-Generator Copy Mechanism**. This allows our network to dynamically copy English loanwords with 100% fidelity while accurately normalizing colloquial Telugu morphemes to proper Unicode script. We achieved an **8x error reduction** (5.37% CER, 79.84 BLEU) on our NVIDIA RTX 4060 GPU with sub-10ms latency, packaged into an enterprise Web Studio and a Chrome Extension."

6.2 Live Presentation Script (10 Slides)

- Slide 1 (Title):** Introduce topic, team, course (AAN), and GPU hardware acceleration.
- Slide 2 (The Problem):** Show examples of Tanglish. Explain why Google Translate fails (loanword corruption).

- **Slide 3 (Dataset Scaling):** Detail the 25,803 pairs from Google Dakshina, Aksharantar, and zero-leakage hash splitting.
- **Slide 4 (Architecture):** Show the 2-Layer BiGRU + Scaled Dot Attention block diagram.
- **Slide 5 (Copy Mechanism):** Explain the p_{gen} gate formula and how it preserves English words.
- **Slide 6 (Benchmark Results):** Present the quantitative comparison table (5.37% CER vs 48.2% baseline).
- **Slide 7 (OmniProcessor):** Highlight the 4 simultaneous target modalities.
- **Slide 8 (Web Studio Demo):** Show live browser interface with TTS audio, mic input, and keyboard drawer.
- **Slide 9 (Chrome Extension):** Demonstrate context menu normalization on WhatsApp Web / Twitter.
- **Slide 10 (Conclusion & Viva):** Summary of contributions and future multi-dialect roadmap.

6.3 Anticipated Technical Viva Questions & Bulletproof Answers

Q1: Why did you choose a 2-Layer BiGRU instead of a Transformer or Large Language Model (LLM)?

Answer: Large Language Models (like LLaMA or GPT-4) contain billions of parameters and take 300ms to 2 seconds per inference, requiring cloud clusters. For real-time keyboard typing and browser extensions, latency must be under 10ms. Furthermore, Transformers lack recurrent inductive biases and require hundreds of thousands of sequences to learn character-level alignments effectively. Our 2-Layer BiGRU has 3.05M parameters, runs in **~8.2 ms on our RTX 4060 GPU**, and achieves superior character alignment on medium-sized corpora.

Q2: How exactly does the Pointer-Generator Copy Mechanism work?

Answer: The Pointer-Generator network computes a scalar generation probability $p_{\text{gen}} = \sigma(W_{\text{ptr}}[s_t; c_t; e(y_{t-1})] + b_{\text{ptr}})$. The final probability distribution is: $P(w) = p_{\text{gen}} P_{\text{vocab}}(w) + (1 - p_{\text{gen}}) \sum_{i: x_i = w} \alpha_{t,i}$. When encountering English loanwords, the network drives $p_{\text{gen}} \rightarrow 0$ and uses the attention weights $\alpha_{t,i}$ as a pointer to copy Latin characters directly from the source sequence into the output.

Q3: What is "Zero-Leakage Splitting" and why is a random train/test split invalid here?

Answer: In code-mixed transliteration, one clean Telugu target sentence often has multiple noisy Romanized variants (e.g., *chala, chaala, chaalaa*). If you perform a naive random split, variant A lands in train and variant B in test. The model simply memorizes the target, producing artificially high BLEU scores. Our zero-leakage hash splitting groups all variants by target lemma and hashes them using SHA-1. All variations of a sentence or vocabulary root are guaranteed to remain strictly in train or strictly in test, never both.

Q4: Why Character-Level Tokenization over Byte-Pair Encoding (BPE)?

Answer: In informal social media Tanglish, users elongate characters for emotional emphasis (e.g., *chaaaaala, plzzzzz*). Subword tokenizers (like BPE) fragment these unseen patterns into rare subwords or `<unk>`, destroying the word. A character tokenizer handles arbitrary character elongation with a compact vocabulary of ~180 tokens and achieves a 0.0% Out-of-Vocabulary rate.

Q5: How do you measure the performance of your system?

Answer: We evaluate across four complementary metrics: 1. **Character Error Rate (CER)**: Levenshtein distance at character level (achieved 5.37%). 2. **chrF**: Character n-gram F-score capturing morphological correctness (achieved 91.29). 3. **BLEU**: Modified n-gram precision with brevity penalty (achieved 79.84). 4. **English Loanword Accuracy**: Verifies that English words are preserved with 100% fidelity without phonetic corruption.

Q6: How is the sub-10ms inference latency achieved in production?

Answer: Three optimizations: 1. **PyTorch CUDA FP16**: Automatic mixed precision running on NVIDIA RTX 4060 Tensor Cores. 2. **Vectorized Beam Search**: Batch decoding executed as tensor matrix operations rather than slow Python loops. 3. **VRAM Model Caching**: Pre-warmed model weights held in GPU memory via the singleton `ModelRegistry`, eliminating disk I/O during API calls.